

h_da

hochschule
darmstadt

member of

-eul+

Faculty of Media

Graphics Programming

Exam Submission

Summer Semester 2026

N-Gin

A first contact with DirectX11

Author: Finn Osterodt

Matriculation: 1139398

Degree: B.A. Animation and Game

Coaches: Prof. Stephan Jacob

Submission: August 31, 2026

Darmstadt

Table of Contents

1	Introduction	2
2	Technical Concept	2
2.1	Engine and ECS Structure	2
2.2	Geometry, Materials and Rendering	3
2.3	Instancing, Culling and Flocking	3
3	Parameters and Controls	4
4	Reallife Usecases	5
5	Challenges	5
5.1	Win32 Interface and Message Handling	5
5.2	Incorrect light direction	5
5.3	Tangent-space Normal Mapping	6
6	Future Work and Current Limitations	6
	References	7
	Image Sources	7
	AI Usage	8

1 Introduction

N-Gin is a small 3D engine and graphics-programming demonstrator written in C++ and HLSL with DirectX 11 on Windows. The project began with the minimal DirectX example introduced in the course, which rendered a single manually defined 2D triangle. I used the final project to extend this starting point step by step into a complete real-time scene and, more importantly, to understand how the individual stages of a graphics pipeline interact in practice. The resulting application combines mesh import, materials and texture maps, real-time lighting, camera projection, GPU instancing, view-frustum culling, an entity-component system (ECS), a simple editor-style interface, procedural flocking behaviour, and frame capture.

The demonstration scene contains 100 instances of a self-modelled teddy-bear mesh moving across a textured ground plane. Their motion is based on the classic flocking model by Craig Reynolds [1] and on autonomous-agent concepts described by Daniel Shiffman [6, 2]: local rules for separation, alignment, and cohesion generate coordinated group behaviour without a central controller. The room uses wrapped boundaries so that agents leaving one side re-enter on the opposite side. This scene was chosen because it simultaneously demonstrates dynamic behaviour and the renderer's ability to process many instances of the same geometry.

2 Technical Concept

2.1 Engine and ECS Structure

Game objects are represented as entity IDs whose data is stored in independent components such as Transform, MeshRenderer, Physics, Camera, Light, Flocking, and VirtualRoom2D. The ECS design was inspired by Sander Mertens' articles on entity-component systems and archetypes [3, 4]. Entities with the same component signature are grouped into archetypes. Adding or removing a component moves the affected entity data to the corresponding archetype, while `ForEach` queries only iterate over archetypes that contain all requested component types. This allows rendering and simulation code to operate on groups of compatible entities without object-specific inheritance structures.

2.2 Geometry, Materials and Rendering

The renderer uses the Direct3D 11 programmable pipeline with a depth-stencil buffer and 4x multisample anti-aliasing. Meshes are imported with the ufbx library [5]. Polygonal faces are triangulated and converted into the engine's own vertex/index representation before GPU buffers are created. Each vertex stores position, normal, UV coordinates, and a tangent. Tangents are derived from triangle edges and UV gradients, re-orthogonalised against the vertex normal with Gram-Schmidt orthogonalisation, and extended by a handedness sign so that the bitangent can be reconstructed in HLSL. Degenerate UV triangles receive a fallback tangent. This tangent basis is required for tangent-space normal mapping.

Materials reference albedo, normal, smoothness, and height texture slots that use anisotropic filtering and additionally store a base colour plus UV tiling and offset. The HLSL shader implements textured Phong-style lighting with ambient, diffuse, and specular components. Tangent-space normal mapping perturbs the surface normal with a normal map, while parallax mapping uses the height map and the view direction in tangent space to offset the texture coordinates before sampling. Directional, point, and spot lights are supported, including distance attenuation for point and spot lights and angular falloff for spotlights. Together, these techniques provide visibly richer surface detail without increasing the geometric complexity of the imported meshes.

2.3 Instancing, Culling and Flocking

To reduce draw-call overhead, visible MeshRenderer entities are grouped by their mesh-material pair. Per-instance world matrices are uploaded to a dynamic instance buffer, and each batch is rendered with DrawIndexedInstanced. Before an entity is added to a batch, its local axis-aligned bounding box is transformed into world space and tested against the camera frustum; objects that are completely outside the frustum are omitted from the render submission.

The flocking simulation runs in a fixed-step update. For every agent, neighbours inside an awareness radius contribute to separation, alignment, and cohesion steering forces. The weighted result is constrained to the horizontal XZ plane and applied as linear acceleration. A separate angular-velocity calculation turns each teddy towards its current movement direction, and quaternion integration updates its orientation.

3 Parameters and Controls

The interface deliberately remains small because the project prioritised the graphics and engine implementation. The Win32 application is divided into a hierarchy, a DirectX render viewport, and an inspector. Selecting an entity in the hierarchy synchronises its transform with the inspector. Position and rotation can be edited either by typing numerical values or by dragging horizontally over the corresponding input field. When the main camera is selected, its field of view, near plane, and far plane can also be modified.

Parameter/Control	Current Location/Value	Effect on result
Transform position/rotation	typed or mouse-dragged [Inspector & Hierarchy]	Moves and rotates the selected entity in real time.
Camera	FOV, near plane, far plane	Changes perspective projection and visible depth range.
Flocking	maxSpeed 8; maxForce 2; awareness 12; separation radius 5 [Flocking.h]	Controls movement speed, steering strength, neighbour detection, and personal-space distance.
Flocking weights	separation 1.5; alignment 1.0; cohesion 0.4[Flocking.h]	Changes the balance between avoidance, shared direction, and attraction to the group.
Flocking-Rotation	turnSpeed 4 rad/s; turnResponse 4 [Flocking.h]	Controls how quickly a teddy rotates towards its velocity direction.
Materials	baseColor; albedo/normal/height maps; UV tiling/offset [CreateMaterials(), N-Gin.cpp]	Changes surface colour and high-frequency surface detail; the carpet uses 10 x 10 tiling.
Lights	type, colour, intensity, range, cone angles [AddLights(), N-Gin.cpp]	Controls illumination, attenuation, and spotlight shape.
Scene setup	CreateMaterials(), AddCamera(), AddLights(), InitGraphics() [N-Gin.cpp]	Defines assets, initial camera/light configuration, and object placement.

4 Reallife Usecases

The project is primarily a learning and demonstration system rather than a production-ready game engine. Nevertheless, its individual systems are reusable. The ECS and flocking logic could form the basis of crowd or animal-group simulations, while instanced rendering is suitable for scenes that contain many copies of the same mesh. With further refactoring, collision handling, asset tooling, and a more complete material model, N-Gin could also serve as the technical foundation for a small independent game or as an educational example for DirectX 11 rendering and data-oriented engine architecture.

5 Challenges

5.1 Win32 Interface and Message Handling

One challenge was the event-driven structure of the native Windows interface. Unlike UI systems I had previously worked with, input and control behaviour is distributed across messages such as `WM_COMMAND`, `WM_MOUSEMOVE`, and `WM_PAINT`. Individual controls are represented by `HWND` handles and require explicit creation and state management. I addressed this by centralising entity selection in the hierarchy, synchronising the inspector through `UpdateInspector()`, and subclassing the numerical edit controls so that the same fields support both text input and Unity-like horizontal dragging. ChatGPT was consulted while learning the Win32 message model and exploring implementations; the resulting code was reviewed and adapted to the project's own state and ECS structures.

5.2 Incorrect light direction

A graphics-specific debugging problem became visible after the arcade-carpet ground plane [7] was added. The ground appeared much darker than expected even though its albedo texture was bright. This made it easier to recognise that the directional-light vector in the shader had the wrong sign, causing illumination to arrive from the opposite direction and effectively light the back side of the surface. Correcting the direction calculation restored the intended

shading and also reinforced the importance of maintaining a consistent convention for light vectors between the CPU-side Transform and HLSL calculations.

5.3 Tangent-space Normal Mapping

Normal mapping introduced another technical issue because imported geometry cannot be assumed to provide a tangent basis in the exact form required by the shader. The engine therefore constructs tangents during mesh import, orthogonalises them against the normal, stores handedness for bitangent reconstruction, and supplies a fallback for degenerate UV triangles. This prevents undefined tangent vectors from destabilising the normal-mapped result.

6 Future Work and Current Limitations

The main architectural limitation is that N-Gin.cpp still contains a large amount of rendering, simulation, scene setup, and Win32 interface logic. Splitting these responsibilities into dedicated renderer, systems, scene, and editor modules would improve readability and reusability. Component behaviour such as physics integration and flocking should likewise be moved into explicit systems instead of remaining closely coupled to the main update functions.

The current material system already combines albedo texturing, tangent-space normal mapping, height-based parallax mapping, and specular reflection. A logical next step would be to extend this material model with shadow mapping and a more physically based treatment of surface properties such as roughness and metallic response. Screen-space post-processing effects, for example ambient occlusion, could further improve depth perception and the visual quality of the final image.

The video recorder also requires more deterministic timing. It currently tells FFmpeg to encode at 60 frames per second but captures once per Render() call, whose frequency is not guaranteed to be 60 Hz. Simply moving capture into the present FixedUpdate() would not solve this, because that update currently runs at 16 Hz. A better solution would be a dedicated 60 Hz capture accumulator or an offline fixed-rate rendering mode in which simulation, rendering, and frame export advance deterministically.

References

- [1] Craig Reynolds. *Boids: Background and Update*. June 29, 1995. URL: <https://www.red3d.com/cwr/boids/> (visited on 08/30/2026).
- [6] Daniel Shiffman. *Coding Challenge #124: Flocking Simulation*. YouTube video. Dec. 11, 2018. URL: <https://www.youtube.com/watch?v=mhjuuH16qHM> (visited on 08/27/2026).
- [2] Daniel Shiffman. *The Nature Of Code. Simulating natural systems with Processing*. D. Shiffman, 2012. Chap. 6: Autonomous Agents. ISBN: 9780985930806.
- [3] Sander Mertens. *Building an ECS #1: Where are my Entities and Components*. Aug. 6, 2022. URL: <https://ajmmertens.medium.com/building-an-ecs-1-where-are-my-entities-and-components-63d07c7da742> (visited on 07/03/2026).
- [4] Sander Mertens. *Building an ECS #2: Archetypes and Vectorization*. Mar. 14, 2020. URL: <https://ajmmertens.medium.com/building-an-ecs-2-archetypes-and-vectorization-fe21690805f9> (visited on 07/03/2026).
- [5] n.d. *"ufbx"*. URL: <https://ufbx.github.io/> (visited on 07/31/2026).

Image Sources

- [7] Luckius. *Arcade Carpet Textures - arcade_carpet_3_512.png*. Jan. 29, 2022. URL: <https://opengameart.org/content/arcade-carpet-textures-arcadecarpet3512png> (visited on 08/30/2026).

AI Usage

ChatGPT by OpenAI and *DeepL* was used for grammar correction and spell checking for this documentation/report.

As part of the DirectX11 engine project, *ChatGPT* by OpenAI was used for code implementation, debugging and general research into engine and shader functionalities/concepts. No AI-generated code was blindly copied and pasted; instead, everything was reviewed, checked for functionality and often adapted to specific contexts, as the AI code served as a starting point for solving a more complex problem. Additionally, comments were added to underline the usecase and enable better understanding for later code-viewers.